Course on Knowledge Graphs in the era of Large Language Models

Laboratory 4: Exposing Tabular Data as an RDF-based Knowledge Graph

Ernesto Jiménez-Ruiz

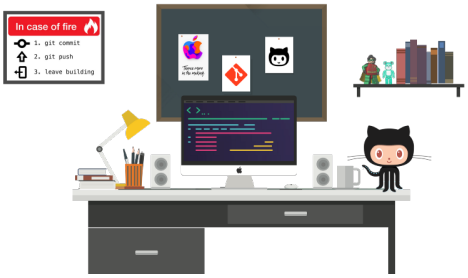
March 3, 2026

Contents

1 Git Repositories

Support codes for the laboratory sessions are available in *github*. There is always a `requirements.txt` within each lab folder. It is recommended to use environments, but it is not strictly necessary.

`https://github.com/city-knowledge-graphs/kg-course-valgrai`



2 Dataset

In this lab session, we will use a dataset about world cities, more specifically, we are using the free subset of the World Cities Database.[1] The dataset is also available in the GitHub repositories:

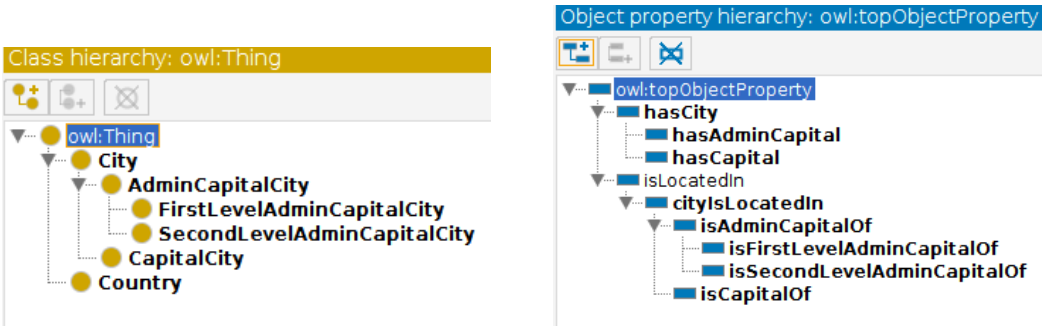
- Full dataset: `worldcities-free.csv`
- 100 rows only: `worldcities-free-100.csv`
- 100 rows only (in Excel): `worldcities-free-100.xlsx`

2.1 Associated ontology

As we saw in the lecture notes, the direct CSV-to-RDF transformation does not properly captures the semantics of the data. The world cities dataset is simple but one could already think about a smart transformation to indicate that the elements in the first column are cities and that cities are located in a country. I have created a simple ontology capturing the domain of the world cities dataset: `ontology_worldcities.owl` and `ontology_worldcities.ttl`. Note that the ontology is relatively simple and it contains a few instances as a proof of concept.

About the ontology. The most important modelling choice is the definitions of different types of `City` (see Figure 1a) and the related object properties (see Figure 1b). I have also defined inverses and a hierarchy of object properties to enhance reasoning (recall the inference rules). The ontology also contains some example instances to test the inferences from Protégé (see Figure 2).

[1] `https://simplemaps.com/data/world-cities`



(a) Class hierarchy.

(b) Object property hierarchy.

Figure 1: Ontology for the world cities dataset.

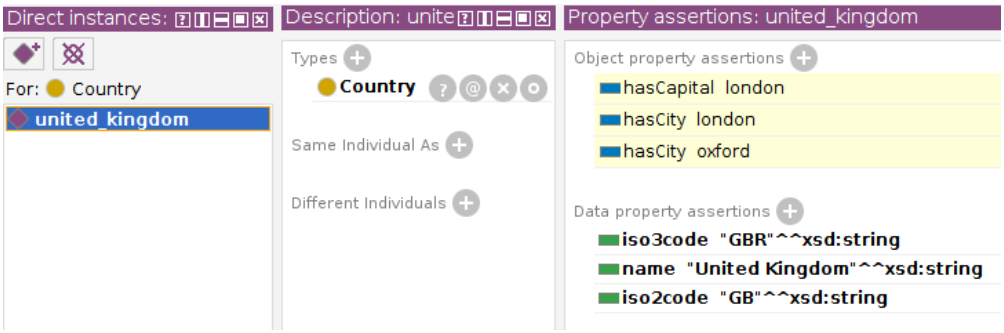


Figure 2: Example data and entailments.

3 Towards 4 ⋆ data

One of the steps to obtain 5-star and FAIR data, as we have seen in the lecture, is to:

- provide unique identifiers to the elements of the data;
- transform the data into RDF triples; and
- describe the data using an ontology.

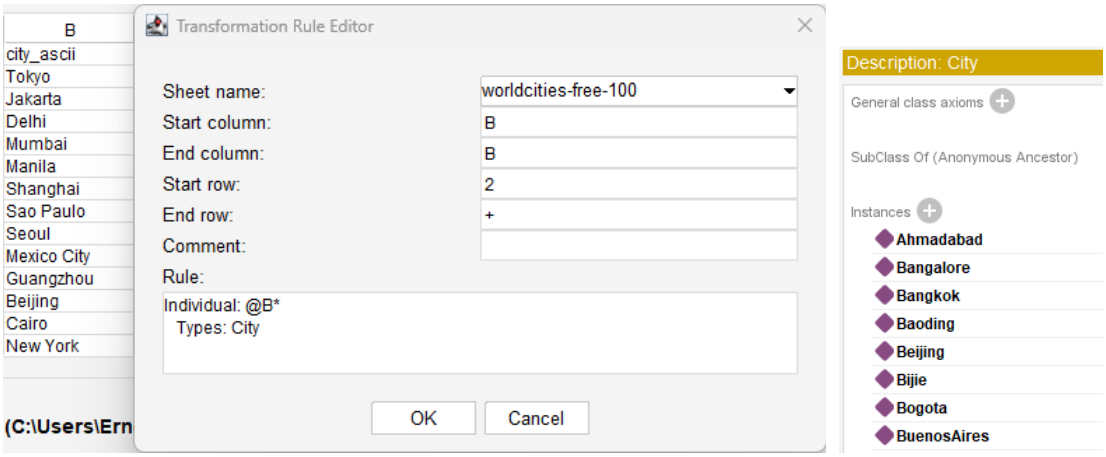
3.1 Transformation programmatically

Support codes: Check **Task 1.1** model solution in lab session 1. This solution follows a semi-automatic approach, that is, some "manual" assumptions have been made about column types and relationships among columns (*i.e.*, mapping/link to the ontology vocabulary), then the transformation to RDF triples has been made automatic.

Task 1: Convert the World Cities CSV file into RDF triples using the vocabulary of the provided ontology (*i.e.*, `ontology_worldcities`) as shown in the examples of Table 1. Create fresh entity URIs for the cities and countries (*e.g.*, `wco:london`, being `wco:` the selected prefix of the namespace defined for the World Cities ontology and data).

```
wco:london  rdf:type              dbo:City.
wco:london  wco:latitude          "51.5072"^^xsd:float.
wco:london  dbo:populationTotal   "10979000"^^xsd:long.
```

Table 1: Example triples.



(a) Cellfie rule. **(b)** Resulting axioms.

Figure 3: Example simple Cellfie rule and its results

3.2 Transformation via Protégé's Cellfie plugin

The Cellfie plugin comes together with the latest Protégé versions (>5.0). Install Protégé if you have not done it already (https://protege.stanford.edu/).

The Cellfie plugin allows for transforming Excel Spreadsheets (apparently, no other format is allowed) into triples/axioms. This is done via some custom mapping rules defined in Protégé's interface.

The language may not be the most intuitive one, depending on your background, but with examples, it may be useful to automate the creation of basic RDF triples from a Spreadsheet with a clear structure.

As a running example, we are using the World Cities dataset (the Excel version worldcities-free-100.xlsx) and its associated ontology (either the .owl or .ttl file): ontology_worldcities.owl. Follow these steps to create instances of the class City:

- Load the ontology_worldcities.owl ontology into Protégé.
- Go to the Protégé menu *"Tools/Create axioms from Excel workbook..."* and select worldcities-free-100.xlsx.
- Add a new transformation rule. You will need to indicate the columns that will be used, and the start (2 in our case) and end rows ("+" to use all rows). Our first rule will only use elements in column B. See rule in Figure 3a.
- *Generate axioms* and add them to the current ontology. The results are shown in Figure 3b.

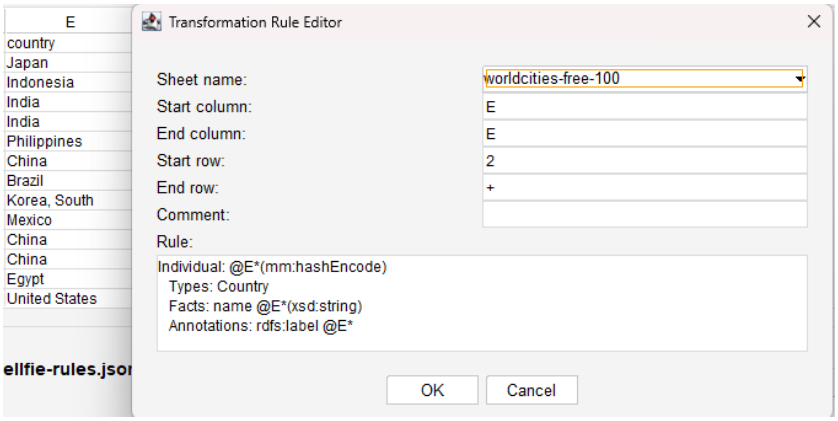


Figure 4: Example of Cellfie rules with unique ids in the URIs

The rule in Figure 3a iterates over all the elements in column B (*i.e.*, `@B*`), creating an OWL individual for each of them and stating that they are members of the class `City` in the `ontology_worldcities.owl` (see Figure 3b).

A similar rule can be defined for the countries in column E. As values in some of the cells may contain characters that are not accepted in the URIs, the rule in Figure 4. uses `(mm:hashEncode)` to generate URIs for each of the countries like `http://www.semanticweb.org/ernesto/in3067-inm713/wco/3536be57ce0713954e454ae6c53ec023`. In addition, the rule creates triples with the predicates `wco:name` and `rdfs:label` to associate with each country-instance its name and a label.

Figure 5 shows a bit more complex rule where cities and countries are connected via the predicate `wco:cityIsLocatedIn`. Note that the URIs created with the command `@E*(mm:hashEncode)` in the rule in Figure 4 are the same as the ones in the rule in Figure 5. In this way, both rules refer to the same entities.

Rules can be edited, saved to a JSON file (text format), and loaded if created in previous sessions.

Task 2: Create and execute the rules in Figures 4 and 5.

Task 3: Create rules for the other columns in the dataset to create all the necessary triples/axioms. Save all the created rules.

There is additional documentation on the Web about how to use Cellfie:

- Manual: `https://github.com/protegeproject/cellfie-plugin`
- Mapping language: `https://github.com/protegeproject/mapping-master/wiki/MappingMasterDSL`
- Grocery tutorial: `https://github.com/protegeproject/cellfie-plugin/wiki/Grocery-Tutorial`

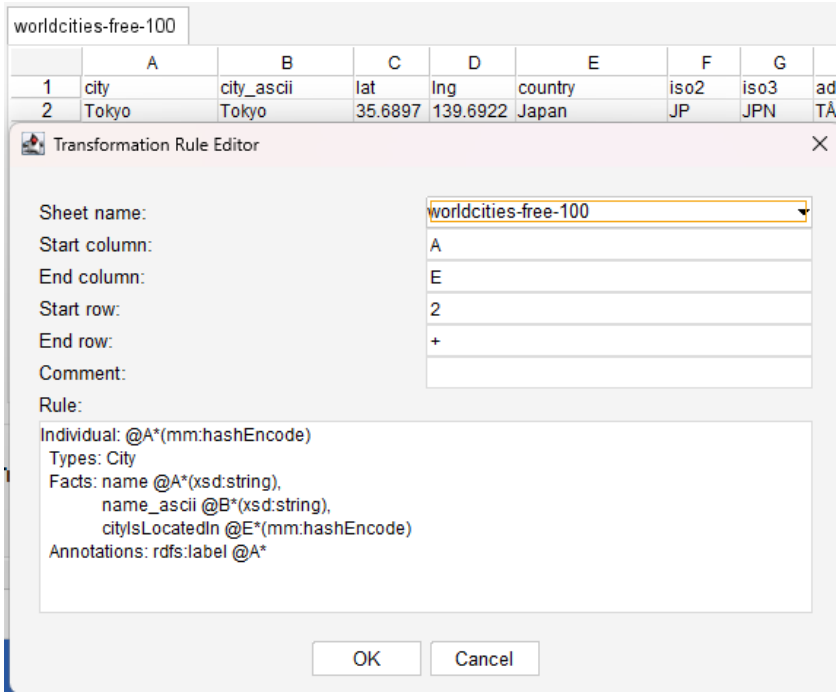


Figure 5: Example of Cellfie rules with unique ids in the URIs and connecting cities with countries.

4 Towards 5 ⋆ data

Linking your data to state-of-the-art KGs will increase its FAIRness as it will be more interoperable. These links are also a prerequisite for 5 ⋆ data.

4.1 Linking to external datasets programmatically

Support codes. I have added to the GiHub repositories the following support codes (including a Python notebook invoking the methods below):

Lookup. There are codes to connect to the look-up services of DBPedia, Wikidata and Google's KG. See example codes in: `lookup.py`.

String Similarity. String similarity will be useful to compare cell values to the label of KG candidates. In Python (see `lexical_similarity.py`) we use the `python-Levenshtein` library (see `requirements.txt`) and the ISub[2] method in `isub.py`.

Task 4: Run the support codes to run the similarity metrics and to connect to the *lookup* with services with different inputs.

[2]A String Metric for Ontology Alignment. ISWC 2005: `http://manolito.image.ece.ntua.gr/papers/378.pdf`

Task 5: Same as Task 1, but reusing entity URIs from DBpedia or Wikidata for cities and countries (*e.g.*, `dbr:London` instead of `wco:london`). Use DBpedia's or Wikidata's KG look-up services to extract KG entity URIs. A look-up service will receive a string as input and retrieve a set of candidate KG entities. For example, the string "United Kingdom" will retrieve, among others, the entity `dbr:United_Kingdom` from DBpedia and the entity `wikidata:Q145` from Wikidata.

Tip: *If there are more than one candidate entity, you could either (1) get the top-1 candidate according to the look-up, or (2) apply additional techniques (e.g., lexical similarity, contextual information) to get the best candidate.*

4.2 Automating the column type prediction

Task 6: Extend your system to be able to automatically predict the type of columns about cities and countries.

Tip: *DBpedia's look-up also returns the associated type for the returned candidate entities.*

Task 6: In the GitHub folder `sem-tab-evaluator`, I have uploaded a small portion of the Tough Tables (2T) dataset[3] (used in the SemTab challenge) that targets DBpedia. I have also uploaded the codes to calculate the accuracy in the CEA and CTA tasks (see `CEA_Evaluator.py` and `CTA_Evaluator.py`). Try to use the created system to annotate the provided tables and get the accuracy of the annotations.

Note 1: *You need to create an output similar to the examples given in the folder `system_example`, where each row identifies the source table, row id, column id (only in CEA), and the annotation URI.*

Note 2: *For clarity, I have created one ground truth file per table (see folder `gt`), but in the SemTab challenge, there is typically a single ground truth file for CEA and CTA, respectively, as each row already identifies the source table.*

4.3 Using the tool OpenRefine

OpenRefine is a free, open-source tool for cleaning, exploring, transforming, and linking messy data, especially in spreadsheets (CSV, Excel, TSV). You can download OpenRefine from `https://openrefine.org/`.

The OpenRefine functionality we are going to use in the lab is the *reconciliation*. Reconciliation is the process of linking entries in a dataset to their corresponding records in an external source (*e.g.*, the Wikidata KG), also called record linkage or data matching.

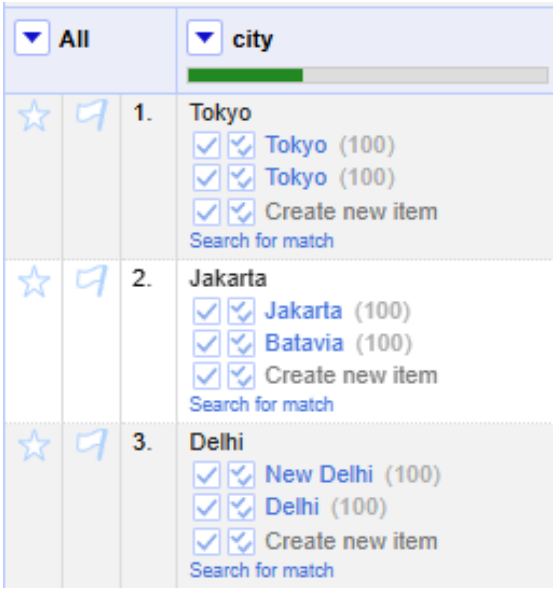
As a running example, we are using the World Cities dataset (the CSV version `worldcities-free-100.csv`). Follow these steps to annotate the columns about cities and countries with entities from the Wikidata KG:

- Download and run OpenRefine (in Windows execute `openrefine.exe`).[4]

[3] Tough Tables (2T): `https://zenodo.org/records/7419275`

[4] Instructions to run OpenRefine: `https://openrefine.org/docs/manual/running`

(a) Cells with a single matched entity.



(b) Cells with multiple entity candidates.

Figure 6: Example of reconciliation with OpenRefine.

- OpenRefine should automatically open a Web browser. Otherwise, you will need to go to the address `http://127.0.0.1:3333/`.
- The first step is to *create a project*: *(i)* Choose the `worldcities-free-100.csv` file and click *Next*; (ii) if the loaded table looks well formatted, then click *Create project* (top right corner).
- Clink of the header of the column *city*, and select *"Reconcile/Start reconciling..."*. Then select the *Wikidata service* and *Next*.
- Next step will be asking to reconcile cells against a type/class from Wikidata. Select `Q515 (city)` and *Start reconciling*. Note that the process may take some time, and not all cells may find a match.
- Some cell may have a single or several entity candidates (see Figure 6). You could manually select the best match for each cell or use an automatic action in the menu *"Reconcile/Actions"*.
- An additional step is to create an additional column with the URL or Id of the matched entities in Wikidata. Select *"Reconcile/Add columns with URLs..."*. The additional column is shown in Figure 6a.
- Finally, you can Export (top right corner) the OpenRefine project in different formats (*e.g.*, as an extended CSV file).

Task 6: Repeat the same process for the column *country*. Save an extended CSV file with two additional columns that contain the URLs of the matched Wikidata URLs for cities and countries, respectively.

There is additional documentation on the Web about how to use OpenRefine:

- Manual: `https://openrefine.org/docs/manual/reconciling`.
- OpenRefine Tutorial: `https://odl.ischool.uw.edu/openrefine_tutorial/`
- OpenRefine Beginner Tutorial (video): `https://youtu.be/wfS1qTKFQoI?si=qv_5cr7_EWnvhtD7`.